

RxJS

Реактивное программирование для фронтенд-разработчиков

От идеи потока до выбора оператора в реальном UI



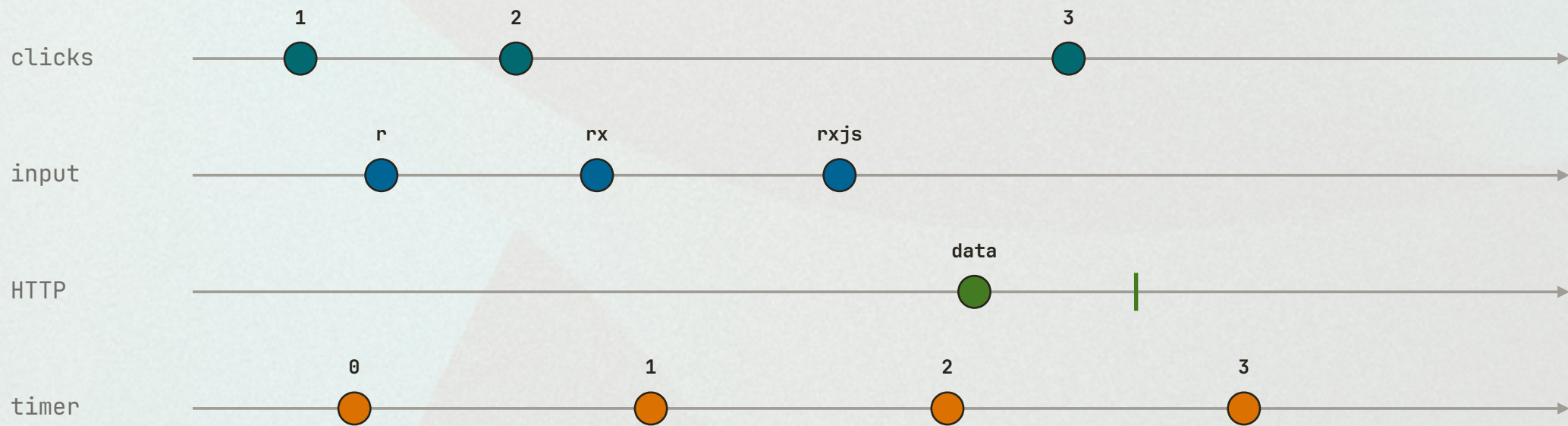
Добро пожаловать в RX

Идея реактивности и разница подходов



Интерфейс — это значения во времени

Экран меняется сам: пользователь печатает, сервер отвечает, таймеры тикают.



Источники разные, природа одна: каждое событие — значение в последовательности.

Почему привычных инструментов недостаточно

Каждый инструмент решает свою задачу. Современный экран — это много событий и состояний одновременно.

Callbacks

Логика распадается на вложенные функции: отменить нельзя, собрать события в одну цепочку — тоже.

Promise

Даёт один результат и не отменяется. А поиск или фильтр — это много результатов на каждое изменение.

Event listeners

Каждый обработчик живёт сам по себе: добавление, удаление и связи между событиями — всё вручную.

Callback, Promise и слушатели работают по отдельности. Интерфейсу нужно связывать события между собой.

Что такое RxJS

Библиотека, которая работает с событиями как с массивами — только значения приходят со временем.

Массив: значения уже есть

Все элементы лежат в памяти. Методы `map` и `filter` обрабатывают их сразу, за один проход.

Поток: значения приходят

Значения появляются одно за другим. Те же `map` и `filter` — но срабатывают на каждое новое значение.

ПРИМЕР: ПОИСК КНИГ

Пользователь печатает



ждём паузу 300 мс



один запрос к API



список книг на экране

RxJS — это `map` и `filter` для событий, которые происходят во времени.

Promise и Observable: в чём разница

Оба получают результат асинхронно. Разница — сколько результатов и когда начинается работа.

Promise — одна посылка

- один запрос — один ответ
- начинает работать сразу после создания
- ответ приходит один раз и навсегда
- остановить выполнение нельзя

Observable — радиозэфир

- значений может быть сколько угодно
- начинает работать при подписке
- новые значения приходят сами
- подписку можно остановить в любой момент

Promise: «жду посылку». **Observable:** «слушаю радио» — включил и получаю, выключил — тишина.

Где живёт Observable

В Angular поток уже встроен во многие повседневные API.

HTTP + Router

HttpClient, paramMap, queryParams

Forms + Events

valueChanges, statusChanges,
fromEvent

Realtime + Time

WebSocket, interval,
пользовательские Subject

Один API помогает одинаково работать с разными асинхронными источниками.

Push и Pull

Кто решает, когда появится следующее значение?

Pull

Потребитель сам запрашивает значение.

```
const value = getData();
```

Push

Источник отправляет значения, когда они появляются.

```
stream$.subscribe(render);
```

Observable использует Push-модель: подписчик реагирует на новые уведомления.

Базовые элементы

Observable, Observer, Subscription и жизненный цикл



Observable выполняется при подписке

Как с функцией: объявление ничего не запускает. Вызов для Observable — это `subscribe()`.

ОБЪЯВЛЕНИЕ → ВЫЗОВ

```
const stream$ = of(1, 2, 3);  
  
// объявили — код не выполнялся  
  
stream$.subscribe(console.log);  
// выполняется сейчас: 1, 2, 3
```

Каждая подписка — новый вызов

Две подписки к одному Observable выполнят инструкцию дважды — как два вызова одной функции.

Нюанс

`from(promise)` наблюдает за задачей, которая уже идёт: Promise начал работу ещё до подписки.

Работа стартует при подписке. Новая подписка — новый запуск.

Жизненный цикл Observable

Контракт допускает много next, затем не более одного завершения.

next(value)

Новое значение. Может произойти 0 или много раз.

error(error)

Аварийное завершение. После него значений нет.

complete()

Успешное завершение. После него значений нет.

Контракт: `next* > (error | complete)?`

Subject и BehaviorSubject

Оба являются горячими источниками и Observer одновременно.

Subject

Отправляет события активным подписчикам.

```
const submit$ = new Subject<void>();  
submit$.next();
```

BehaviorSubject

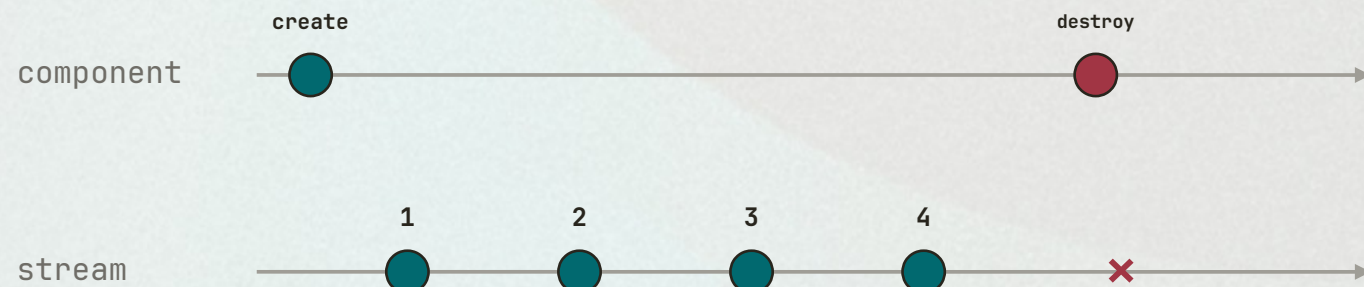
Хранит последнее значение и сразу отдаёт его новому подписчику.

```
const page$ = new BehaviorSubject(1);
```

Subject — для событий. BehaviorSubject — когда новый подписчик должен сразу получить текущее значение.

Subscription — это ресурс

Жизненный цикл подписки должен совпадать с жизненным циклом владельца.



```
const sub = interval(1000)
  .subscribe(render);

sub.unsubscribe();
```

Предпочтительно в Angular

AsyncPipe или takeUntilDestroyed() связывают подписку с компонентом.

unsubscribe прекращает подписку; будет ли отменена сама операция — зависит от источника.

Итоги раздела

Базовые вопросы, на которые мы теперь умеем отвечать — попробуйте ответить сами



Когда Observable начинает работу — при создании или при подписке?



Что происходит с потоком после error или complete?



Чем BehaviorSubject отличается от Subject?



Кто и когда должен вызывать unsubscribe()?

Затрудняетесь с ответом — самое время переспросить, прежде чем идти дальше.

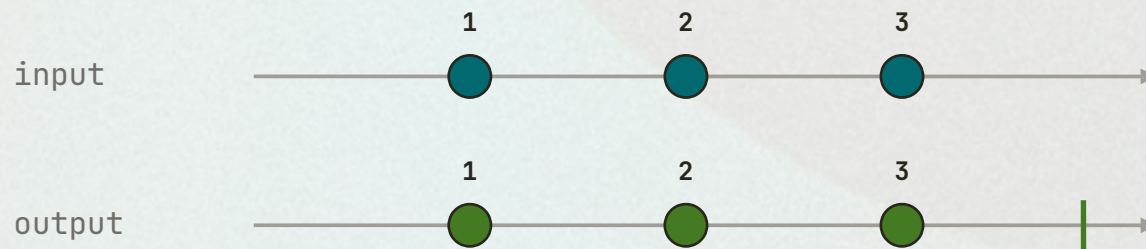
Операторы создания

Как превратить значения, массивы, Promise и время в поток



of()

Создаёт поток из готовых аргументов.



```
of(10, 20, 30)  
  .subscribe(console.log);
```

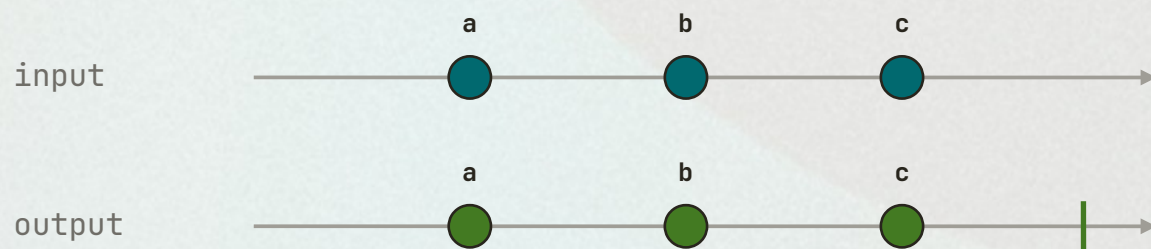
Когда использовать

Тестовые значения, начальные данные, замена ошибки значением в `catchError`.

`of` сохраняет структуру каждого аргумента.

from()

Разворачивает iterable или связывает Promise с Observable.



```
from(['a', 'b', 'c'])  
  .subscribe(console.log);
```

Когда использовать

Массивы, iterable, Promise и сторонние асинхронные API.

`from` превращает элементы коллекции в отдельные next.

of([1, 2, 3]) vs from([1, 2, 3])

Одинаковый массив, разная форма потока.

of([1, 2, 3])

Один next со всем массивом:

– ([1, 2, 3]) – |

from([1, 2, 3])

Три отдельных next:

– 1 – 2 – 3 – |

`of` передаёт массив целиком. `from` разворачивает его элементы.

from(promise)

Результат Promise становится одним next, затем complete.

PROMISE → OBSERVABLE

```
const promise = fetch('/api/users');  
  
from(promise).subscribe({  
  next: response ⇒ render(response),  
  error: showError  
});
```

Нюанс запуска

Promise уже начал работу до вызова from(). Подписка не делает его ленивым.

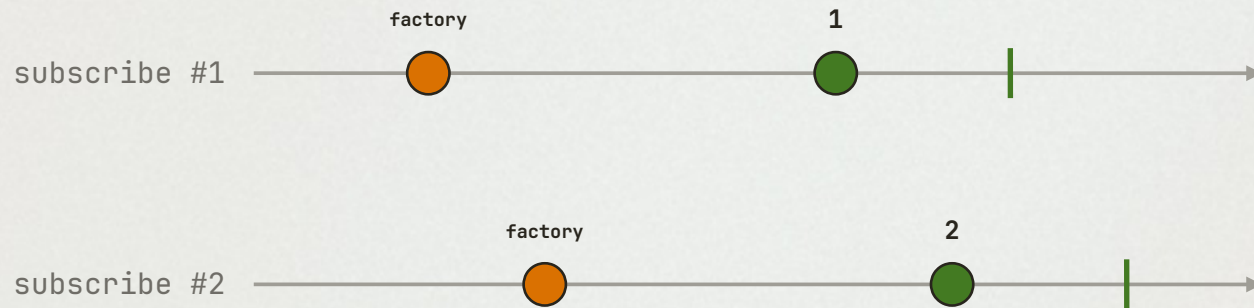
Нужен запуск при подписке?

Об этом — следующий слайд: defer().

resolve > next + complete; reject > error.

defer()

Фабрика вызывается заново при каждой подписке.



```
let n = 0;  
const request$ = defer(  
  () => of(++n)  
);  
  
request$.subscribe(console.log); // 1  
request$.subscribe(console.log); // 2
```

Используйте `defer`, когда параметры или сама операция должны вычисляться в момент `subscribe`.

throwError()

Создаёт поток, который сразу завершается ошибкой.



```
throwError(() => new Error('Boom'))  
  .subscribe({ error: console.error });
```

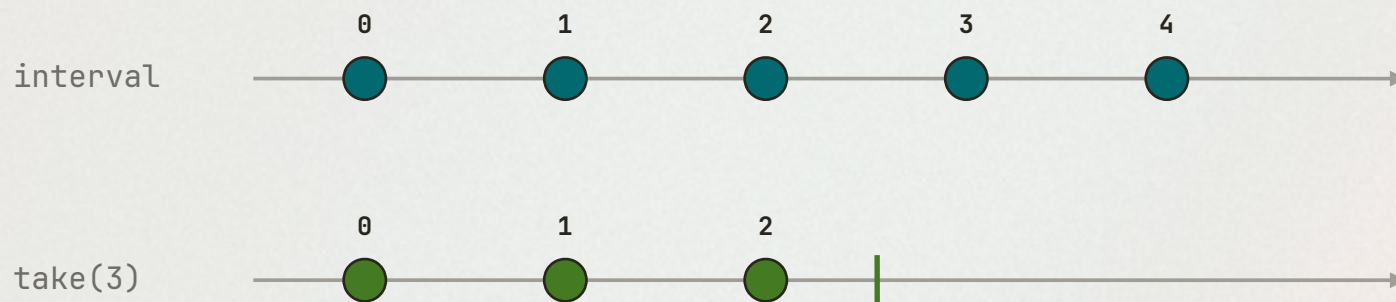
Когда использовать

Повторный выброс ошибки из `catchError` после логирования.

`throwError` передаёт ошибку дальше; `of(...)` в `catchError` заменяет её значением.

interval() + take()

Бесконечный таймер становится ограниченным потоком.



```
interval(1000).pipe(  
  take(3)  
)  
.subscribe(console.log);  
// 0, 1, 2, complete
```

`take(N)` завершает поток после N значений и освобождает подписку.

Итоги раздела

Базовые вопросы, на которые мы теперь умеем отвечать — попробуйте ответить сами

? Чем `of([1, 2, 3])` отличается от `from([1, 2, 3])`?

? Что сделает `from(promise)`, если промис завершился ошибкой?

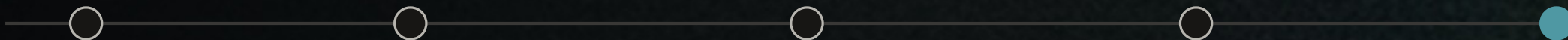
? Сколько раз `defer()` вызывает фабрику — и когда именно?

? Как остановить бесконечный `interval()`?

Затрудняетесь с ответом — самое время переспросить, прежде чем идти дальше.

Pipe и операторы

Трансформация, фильтрация и побочные эффекты



pipe() — читаемый конвейер

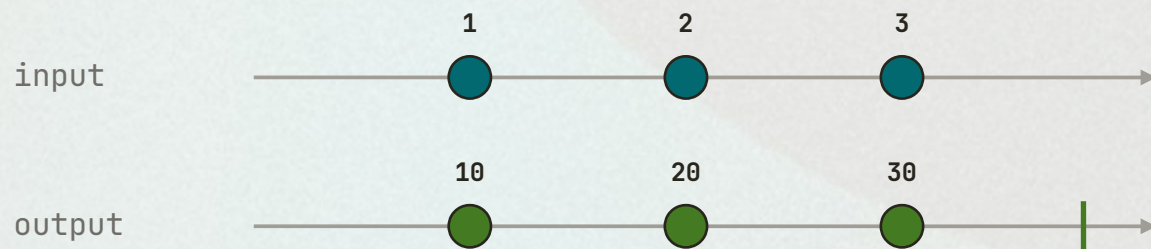
Каждый оператор получает выход предыдущего.



Порядок операторов важен: меняется не только код, но и смысл потока.

map()

Преобразует каждое значение без изменения количества событий.



```
of(1, 2, 3).pipe(  
  map(x => x * 10)  
)
```

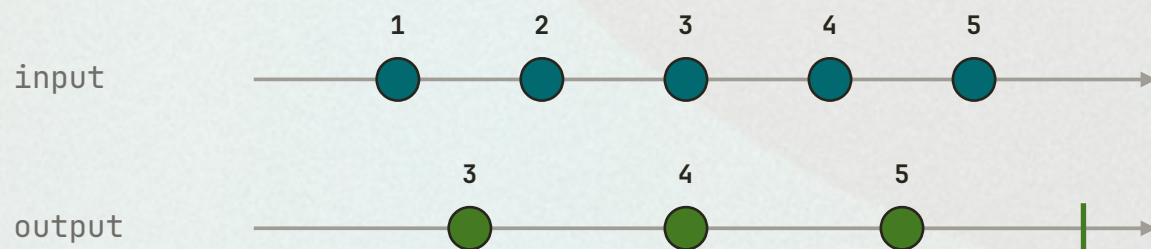
Когда использовать

Преобразование API-модели во ViewModel.

`map` — чистая трансформация: вход > выход.

filter()

Пропускает только значения, удовлетворяющие условию.



```
of(1, 2, 3, 4, 5).pipe(  
  filter(x ⇒ x > 2)  
)
```

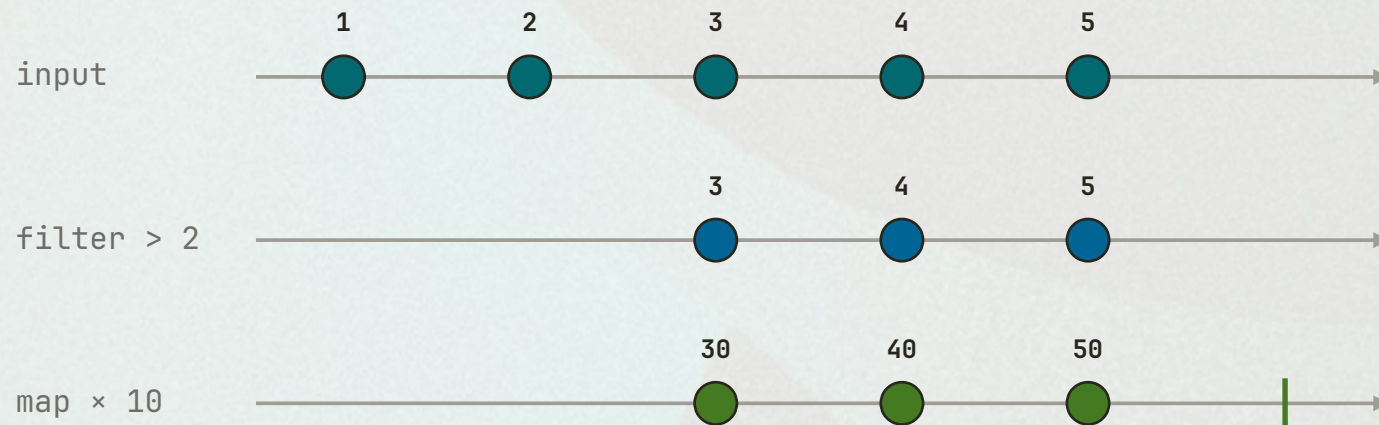
Когда использовать

Отбрасывание невалидных или неинтересных событий.

`filter` не меняет значение, а решает, пропустить ли его.

map + filter

Операторы комбинируются, а порядок определяет результат.

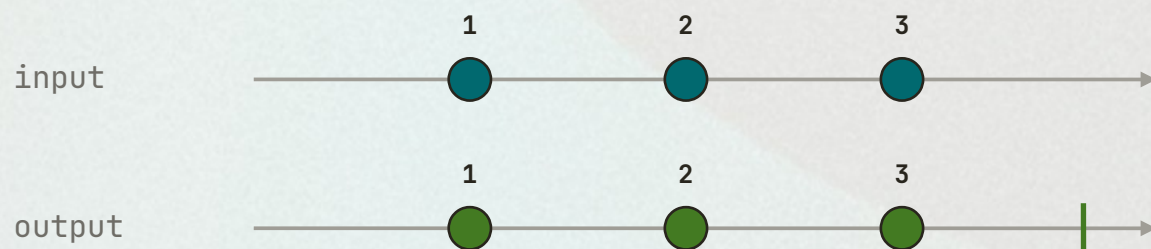


```
of(1, 2, 3, 4, 5).pipe(  
  filter(x => x > 2),  
  map(x => x * 10)  
)
```

`filter > map` и `map > filter` могут давать разные результаты.

tap()

Выполняет побочное действие и не меняет значение.



```
source$.pipe(  
  tap(value => console.log(value)),  
  map(value => value * 10)  
)
```

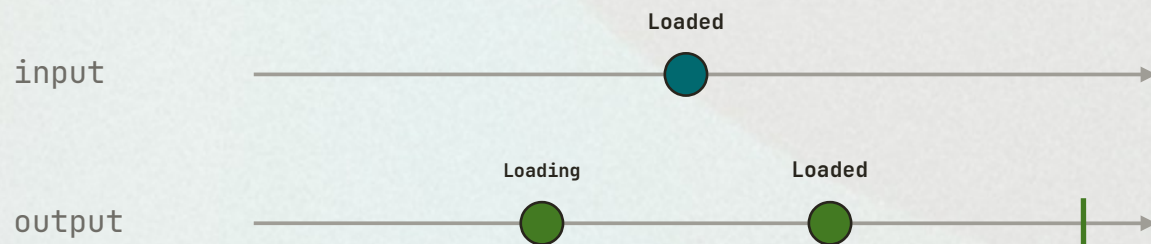
Когда использовать

Логи, диагностика и внешние эффекты без трансформации.

Если нужно изменить значение, используйте `map`, а не `tap`.

startWith()

Добавляет начальное значение перед событиями источника.



```
data$.pipe(  
  startWith('Loading')  
)
```

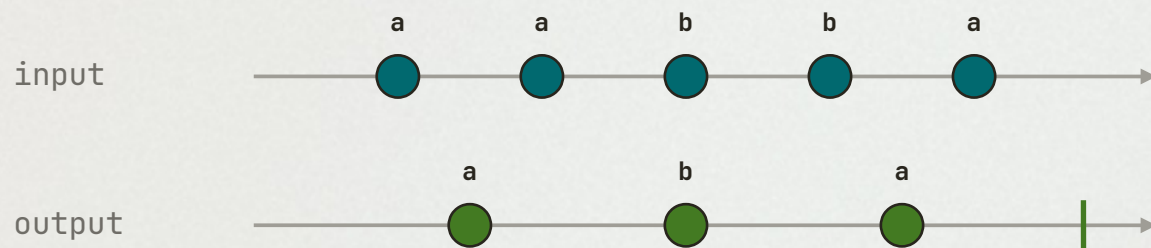
Когда использовать

Начальное UI-состояние до первого ответа.

`startWith` определяет первое значение, которое увидит подписчик.

distinctUntilChanged()

Удаляет только последовательные дубликаты.



```
valueChanges.pipe(  
  distinctUntilChanged()  
)
```

Когда использовать

Не запускать логику повторно, если значение не изменилось.

Для объектов задайте `comparator` или сравнивайте ключ.

debounceTime()

Выдаёт последнее значение после паузы во входном потоке.



```
search.valueChanges.pipe(  
  debounceTime(300),  
  distinctUntilChanged()  
)
```

Что происходит

Таймер перезапускается на каждом новом событии. Выходит только последнее значение после тишины.

``debounceTime`` = «подожди, пока пользователь перестанет печатать».

Итоги раздела

Базовые вопросы, на которые мы теперь умеем отвечать — попробуйте ответить сами



Почему порядок операторов в `pipe()` важен?



Чем `tap()` отличается от `map()`?



Какое первое значение увидит подписчик после `startWith()`?



Какую проблему решает `debounceTime()`?

Затрудняетесь с ответом — самое время переспросить, прежде чем идти дальше.

Higher-order mapping

Четыре стратегии для конкурирующих внутренних потоков



Событие запускает новый Observable

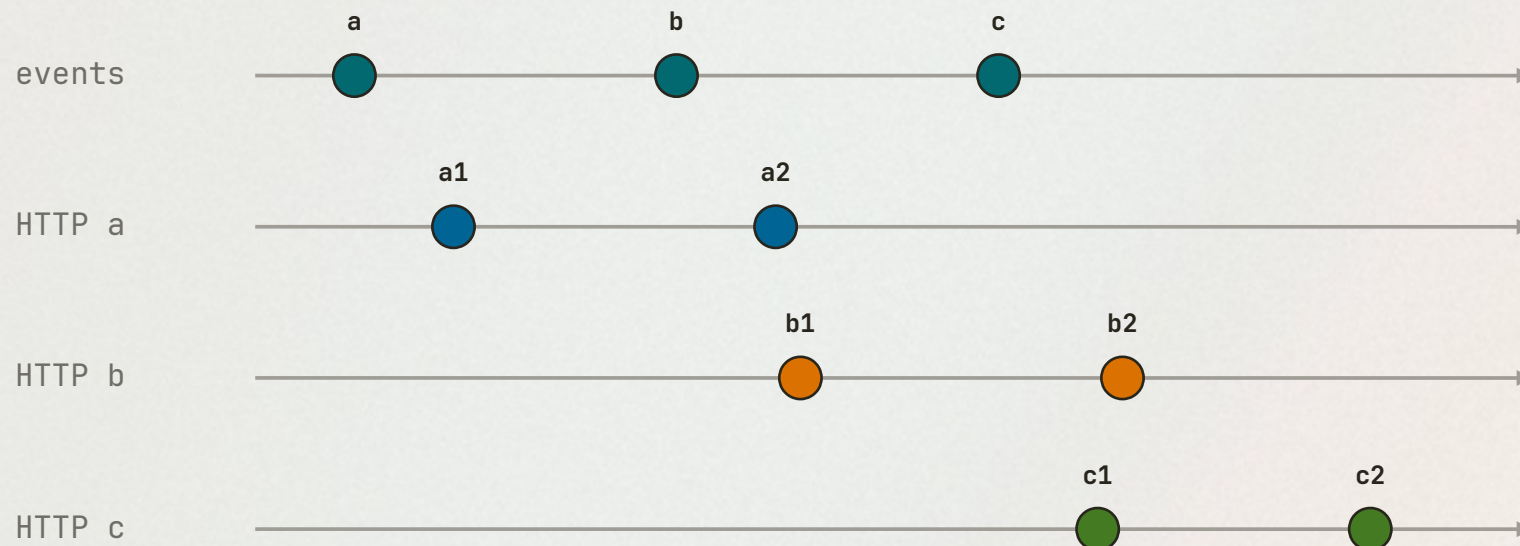
Значение внешнего потока может создавать HTTP-запрос.



Higher-order operator управляет подписками на внутренние Observable.

Что делать, если событий несколько?

Новый запрос начинается, пока предыдущий ещё выполняется.



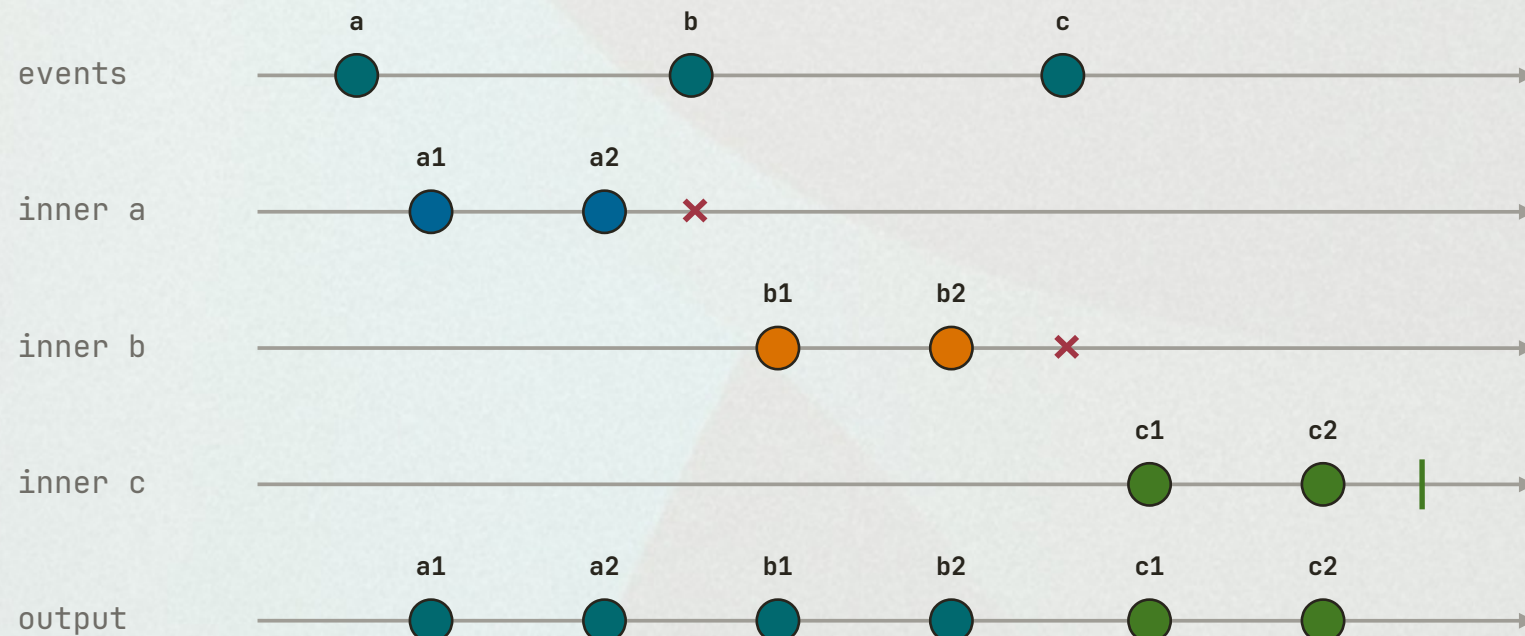
Четыре стратегии

- Отменить
- Параллелить
- Поставить в очередь
- Игнорировать новое

Выбор оператора — это решение о конкуренции, порядке и потере событий.

switchMap — отменить прошлое

Новый input переключает подписку на новый внутренний поток.



Лучше всего

Поиск по мере ввода
Навигация
Устаевающие запросы

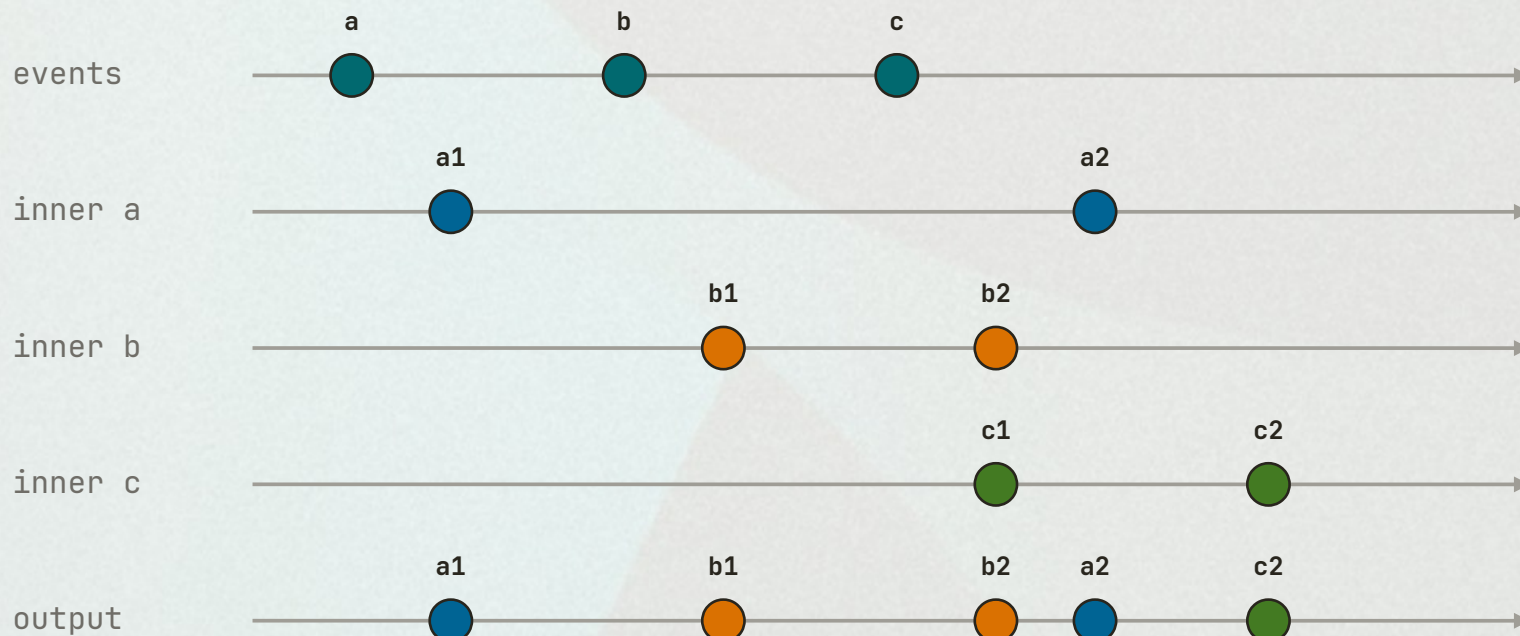
Опасно

Сохранение данных: новый input может отменить важный POST.

`switchMap` = «забудь прошлое, работай с актуальным».

mergeMap — выполнять параллельно

Все внутренние потоки активны одновременно.



Лучше всего

Независимые загрузки
Логи
Фоновые действия

Опасно

Для поиска ответы могут прийти
не в том порядке.

``mergeMap`` = «запусти всё; результаты приходят по готовности».

concatMap — поставить в очередь

Следующая работа ждёт завершения предыдущей.



Лучше всего

Последовательные сохранения
Команды с важным порядком

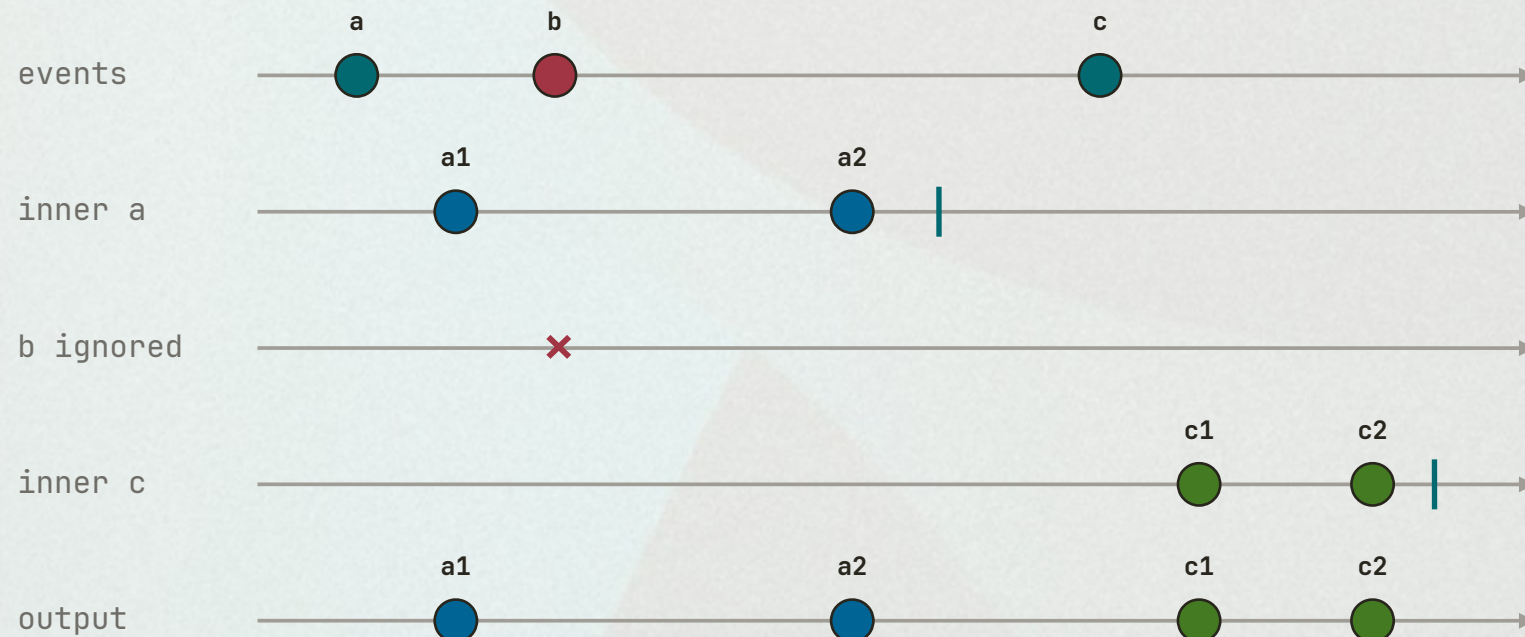
Опасно

Быстрый input создаёт длинную
очередь и задержку.

`concatMap` = «сохрани порядок и ничего не потеряй».

exhaustMap — игнорировать новое

Пока внутренний поток активен, новые события отбрасываются.



Лучше всего

Submit
Login
Защита от двойного клика

Опасно

Пользовательский input может
потеряться без ответа.

``exhaustMap`` = «занят — новые команды не принимаю».

Четыре стратегии на одном экране

Оператор выбирается по продуктовой семантике, а не по привычке.

switchMap

Отменяет прошлое
Главный приоритет: актуальность

mergeMap

Параллелит всё
Главный приоритет: скорость

concatMap

Сохраняет очередь
Главный приоритет: порядок

exhaustMap

Игнорирует новое
Главный приоритет: защита

Сначала сформулируйте, что делать с конкурирующей работой. Затем выбирайте оператор.

Алгоритм выбора higher-order оператора

Четыре вопроса приводят к одному из четырёх решений.

01

Новая работа делает прошлую неактуальной?



switchMap

02

Порядок не важен, можно параллельно?



mergeMap

03

Нужно выполнить всё строго по очереди?



concatMap

04

Повторы нужно игнорировать, пока заняты?



exhaustMap

Итоги раздела

Базовые вопросы, на которые мы теперь умеем отвечать — попробуйте ответить сами



Какой оператор отменяет предыдущий запрос — и когда это опасно?



Почему exhaustMap защищает кнопку «Оформить»?



Когда concatMap лучше mergeMap?

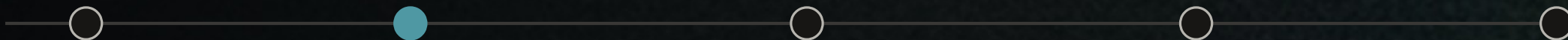


Чем опасен mergeMap для поиска?

Затрудняетесь с ответом — самое время переспросить, прежде чем идти дальше.

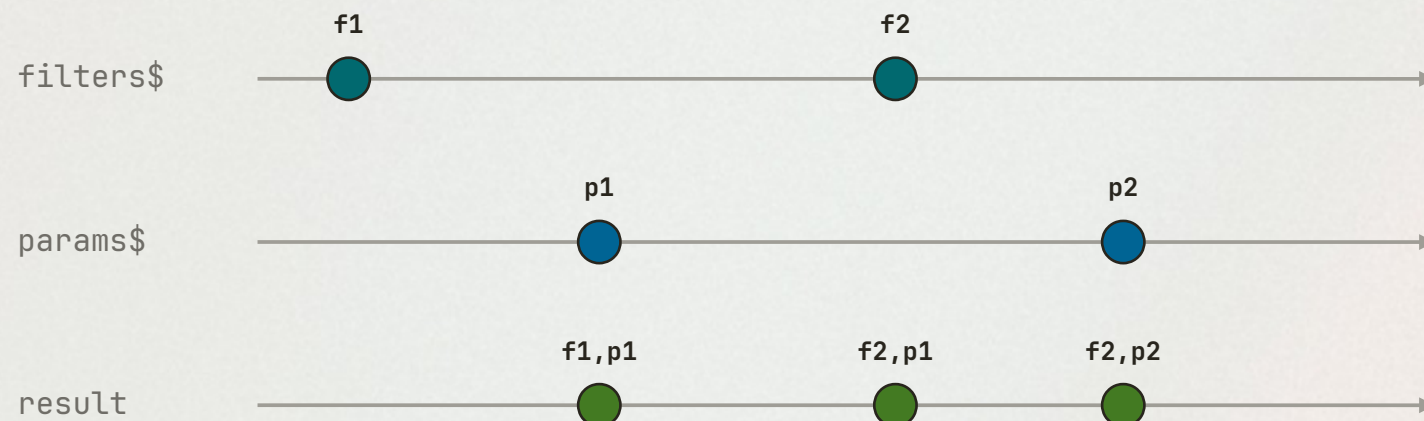
Комбинирование и жизненный цикл

Связать несколько источников и управлять ресурсами



combineLatest()

Выдаёт последние значения при изменении любого источника.

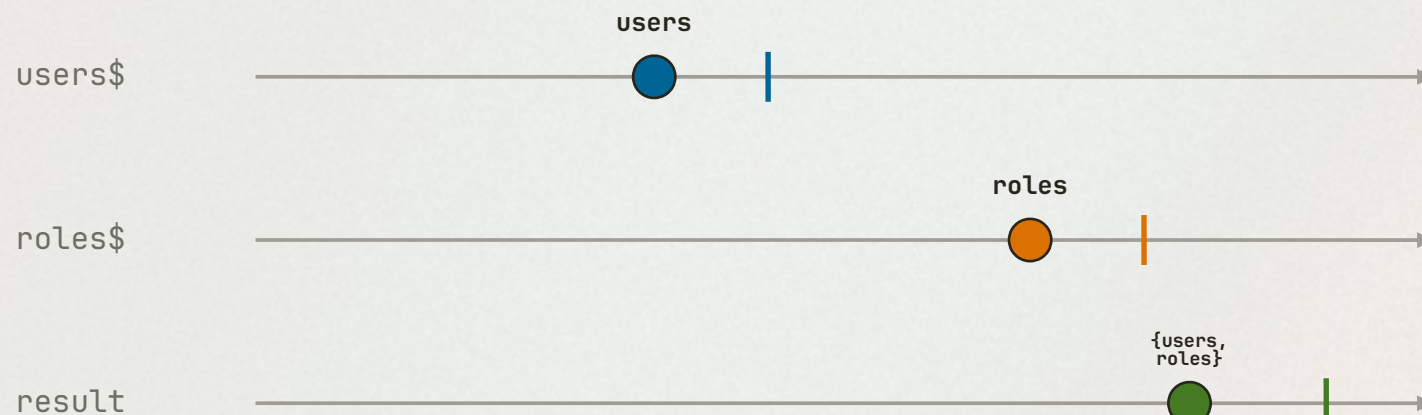


```
combineLatest([
  filters$,
  route.queryParams
])
```

Результата нет, пока каждый источник не отправит хотя бы одно значение.

forkJoin()

Ждёт complete всех источников, затем выдаёт последние значения.



```
forkJoin({  
  users: http.get('/users'),  
  roles: http.get('/roles')  
})
```

Бесконечный источник не завершится — `forkJoin` не выдаст результат.

combineLatest vs forkJoin

Выбор зависит от жизненного цикла источников.

combineLatest

Живые потоки
Много результатов
Реагирует на любое изменение
Ждёт первый next каждого

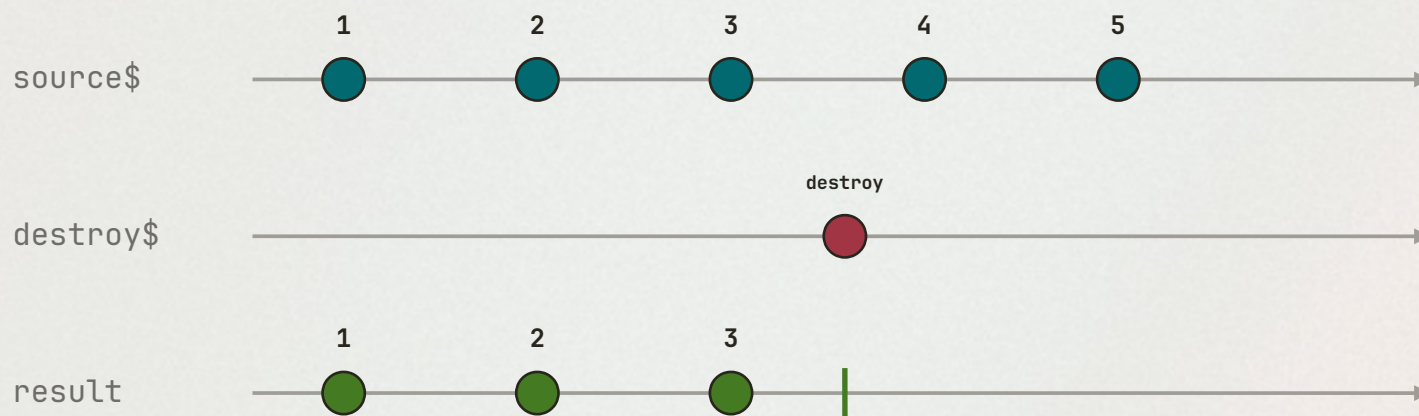
forkJoin

Завершающиеся потоки
Один итоговый результат
Ждёт complete всех
Подходит для группы HTTP

Живые значения > combineLatest. Завершающиеся задачи > forkJoin.

takeUntil()

Завершает поток при первом значении управляющего Observable.

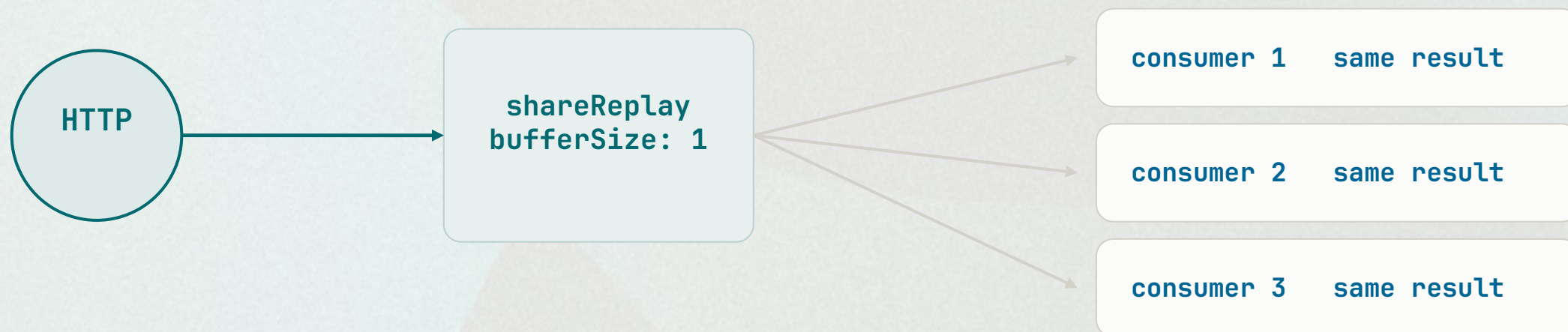


```
source$.pipe(  
  takeUntil(destroy$)  
)
```

В Angular 16+ предпочитайте `takeUntilDestroyed()`.

shareReplay()

Одна подписка на источник, последнее значение доступно новым потребителям.



`shareReplay` не вечный кэш: заранее определите `refCount`, сброс и владельца live-источника.

Итоги раздела

Базовые вопросы, на которые мы теперь умеем отвечать — попробуйте ответить сами



combineLatest или forkJoin — что выбрать для живых фильтров?



Что будет с forkJoin, если один источник не завершится?



Как takeUntil останавливает поток?



Когда shareReplay экономит запросы?

Затрудняетесь с ответом — самое время переспросить, прежде чем идти дальше.

RxJS в Angular

HttpClient, Router, Forms, шаблоны и жизненный цикл подписок



HttpClient возвращает Observable

Один HTTP-вызов обычно выдаёт ответ и завершается.

SERVICE

```
getUser(id: number) {  
  return this.http.get<User>(  
    `/api/users/${id}`  
  );  
}
```

Почему это удобно

- Отмена через unsubscribe
- Композиция с Router и Forms
- Единая обработка ошибок

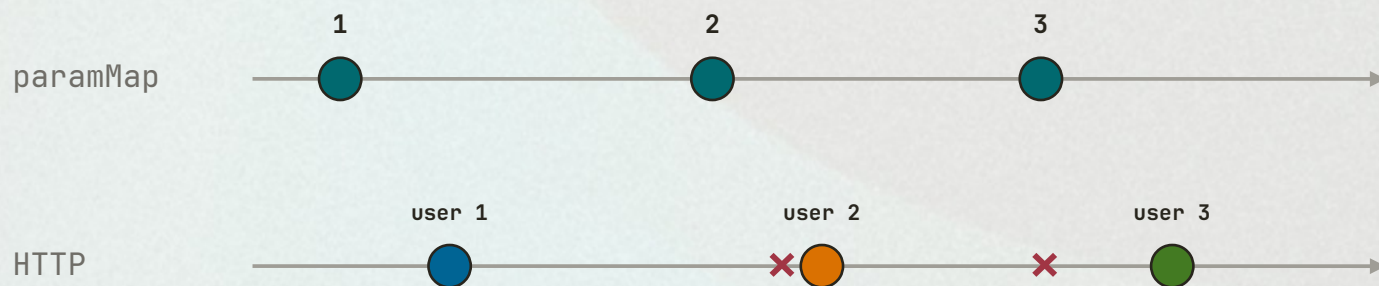
Повтор запроса

Каждая новая подписка запускает HTTP-запрос заново.

HTTP Observable завершается сам, но композиция и владение подпиской всё равно важны.

Router params › актуальные данные

Каждая навигация создаёт новое значение ID.



```
route.paramMap.pipe(  
  switchMap(params =>  
    api.get(params.get('id'))  
  )  
)
```

switchMap отменяет устаревший запрос при смене URL.

Reactive Forms как поток

valueChanges превращает ввод пользователя в Observable.

```
results$ = search.valueChanges.pipe(  
  debounceTime(300),  
  distinctUntilChanged(),  
  switchMap(term ⇒ api.search(term))  
);
```

Смысл цепочки

Пауза › убрать дубли › отменить старый запрос › показать актуальное.

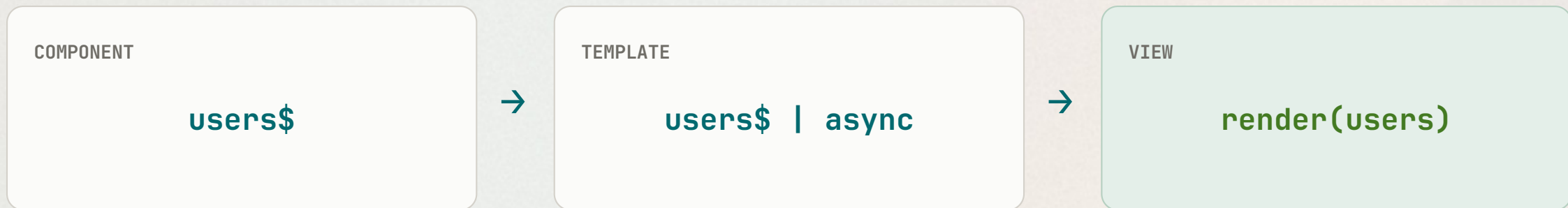
В шаблоне

```
results$ | async
```

Форма сообщает события; конвейер операторов превращает их в состояние экрана.

AsyncPipe управляет подпиской

Подписка создаётся при отрисовке шаблона и очищается при уничтожении компонента.



TEMPLATE

```
@if (users$ | async; as users) {  
  @for (user of users; track user.id) {  
    <app-user-card [user]="user" />  
  }  
}
```

Если значение нужно только шаблону, AsyncPipe обычно проще ручного subscribe.

Очистка подписок в Angular

Ручная подписка (imperative subscribe) — явный вызов `.subscribe()` в коде компонента. Её нужно остановить при уничтожении компонента.

AsyncPipe

Шаблон сам подписывается и отписывается. Для значений, которые нужны только шаблону.

takeUntilDestroyed()

Автоматически останавливает ручную подписку при уничтожении компонента.

takeUntil(destroy\$)

Классический вариант для старых версий Angular.

У каждой подписки есть владелец — и он обязан её остановить.

takeUntilDestroyed и injection context

Вызов без аргумента доступен не в любом месте класса.

В ОБЫЧНОМ МЕТОДЕ

```
private destroyRef = inject(DestroyRef);

ngOnInit() {
  this.api.getLive().pipe(
    takeUntilDestroyed(this.destroyRef)
  ).subscribe();
}
```

Без аргумента

Работает в injection context: например, в инициализации поля или конструкторе.

С DestroyRef

Передайте DestroyRef явно в ngOnInit и других обычных методах.

Короткий API не отменяет необходимость понимать, кто владеет подпиской.

Итоги раздела

Базовые вопросы, на которые мы теперь умеем отвечать — попробуйте ответить сами



Почему HttpClient возвращает Observable, а не Promise?



Как связать смену маршрута с запросом данных?



Что делает AsyncPipe с подпиской?



Где takeUntilDestroyed() работает без аргументов?

Затрудняетесь с ответом — самое время переспросить, прежде чем идти дальше.

Практические паттерны

От продуктовой ситуации к оператору и коду



Поиск по мере ввода

Демо-приложение · поиск в книжном маркетплейсе

Пользователь печатает быстро, а запрос на каждое нажатие клавиши — это шум и лишняя нагрузка на сервер.

`debounceTime(300)`

→

`distinctUntilChanged()`

→

`switchMap`

```
search.valueChanges.pipe(  
  debounceTime(300),  
  distinctUntilChanged(),  
  switchMap(term ⇒ http.get('/api/books', { params: { term  
} })))  
)
```

Частая ошибка

Запрос на каждый символ — или `mergeMap`, ответы которого приходят не по порядку.

Быстрая навигация по каталогу

Демо-приложение · карточка товара

Пользователь листает товары быстрее, чем приходят ответы, — старые ответы уже никому не нужны.

switchMap

```
route.paramMap.pipe(  
  switchMap(params => api.getBook(params.get('id')))  
)
```

Частая ошибка

mergeMap даёт гонку: ответ по старому товару может прийти последним и перезаписать новый.

Submit формы заказа

Демо-приложение · оформление заказа

Двойной клик по кнопке «Оформить» не должен создавать два одинаковых заказа.

exhaustMap

```
submit$.pipe(  
  exhaustMap(() => http.post('/api/orders', form.value))  
)
```

Частая ошибка

mergeMap отправит второй POST — заказ удвоится.

Параллельные запросы

Демо-приложение · торговый дашборд

Дашборду нужны котировки четырёх активов, а грузить их по очереди — слишком медленно.

forkJoin

```
forkJoin(  
  symbols.map(s => http.get(`/api/quotes/${s}`))  
)
```

Частая ошибка

Если хотя бы один запрос не завершится, `forkJoin` не выдаст ничего.

Что делать, если запрос упал

Демо-приложение · карточка профиля

Сервер иногда отвечает ошибкой. Экран не должен ломаться, а пользователь — видеть, что произошло.

retry

→

catchError

```
http.get('/api/profile').pipe(  
  retry({ count: 2, delay: 1000 }),  
  catchError(() ⇒ of(null))  
)
```

Частая ошибка

catchError обязан вернуть Observable:
просто вернуть значение нельзя.

Фильтры и URL вместе

Разбор на лекции

Фильтры в форме и параметры в адресной строке должны всегда давать один и тот же запрос.

combineLatest

→

switchMap

```
combineLatest([filters$, route.queryParams]).pipe(  
  switchMap(([f, p]) => api.load({ ...f, ...p }))  
)
```

Частая ошибка

Два отдельных subscribe: состояние расходится, запросы конкурируют.

Состояния экрана

Демо-приложение · карточка профиля

Пока идёт запрос — каркас-заглушка, затем данные или ошибка. Все три состояния должны быть явными.

startWith

→

map

→

catchError

```
load$.pipe(  
  switchMap(() ⇒ api.getProfile().pipe(  
    map(p ⇒ ({ status: 'data', p })),  
    catchError(e ⇒ of({ status: 'error', e })),  
    startWith({ status: 'loading' })  
  ))  
)
```

Частая ошибка

Один boolean loading не описывает данные и ошибку одновременно.

Утечка подписки

Демо-приложение · музыкальный плеер

Плеер закрыли, а подписка живёт и обновляет экран, которого больше нет.

`takeUntilDestroyed`

```
private destroyRef = inject(DestroyRef);

ngOnInit() {
  player.track$.pipe(
    takeUntilDestroyed(this.destroyRef)
  ).subscribe(render);
}
```

Частая ошибка

Ручной список подписок работает, но его легко забыть пополнить.

Тестирование RxJS-потоков

TestScheduler управляет временем в тесте: паузы вроде `debounceTime(300)` не ждут по-настоящему.

```
scheduler.run(({ cold, expectObservable }) => {  
  // '-a-b|' — компактная запись потока:  
  // буква — значение, - — пауза, | — завершение  
  const result$ = cold('-a-b|').pipe(  
    map((v) => v.toUpperCase())  
  );  
  
  // значения A и B в те же моменты времени  
  expectObservable(result$).toBe('-A-B|');  
});
```

Marble-строка — это поток

«-a-b|» читается так: пауза, значение a, пауза, значение b, завершение.

Что проверяем

Порядок значений, моменты появления и завершение потока — а не только итоговый массив.

Тест выполняется мгновенно: расписание событий задаётся строкой, а не настоящим временем.

Памятка: ситуация › оператор

Сформулируйте задачу словами — оператор найдётся сам.

СИТУАЦИЯ	ОПЕРАТОР
Нужна пауза после ввода	<code>debounceTime</code>
Убрать повторяющиеся значения	<code>distinctUntilChanged</code>
Отменить устаревший запрос	<code>switchMap</code>
Защитить submit от двойного клика	<code>exhaustMap</code>
Сохранить строгий порядок	<code>concatMap</code>
Запросы параллельно	<code>mergeMap</code>
Ждать все запросы разом	<code>forkJoin</code>
Реагировать на любое изменение	<code>combineLatest</code>
Задать начальное состояние	<code>startWith</code>
Автоматическая отписка	<code>takeUntilDestroyed</code>

Как пользоваться

Скажите задачу словами — «нужна пауза», «отменить прошлое» — и найдите строку в таблице.

Проверьте себя

Все десять ситуаций разобраны в демо-приложении — сверьте таблицу с живым кодом.

Четыре вопроса перед выбором

Главное в higher-order операторах — поведение при конкуренции потоков.

Отменять прошлое?

switchMap

Выполнять параллельно?

mergeMap

Сохранять очередь?

concatMap

Игнорировать повторы?

exhaustMap

Сначала выберите поведение при конкуренции. Код оператора станет очевидным.

RxJS за пределами Angular

Тот же RxJS: в React — через rxjs-hooks, в чистом JavaScript — напрямую.

REACT · RXJS-HOOKS

```
// React + rxjs-hooks
const BookSearch = ({ term }) => {
  const books = useObservable(
    (inputs$) => inputs$.pipe(
      map([t] => t),
      debounceTime(300),
      switchMap((t) =>
        api.get(`/api/books?term=${t}`))
    ),
    [], // начальное значение
    [term] // входные данные
  );
  return <List items={books ?? []} />;
};
```

ЧИСТЫЙ JAVASCRIPT

```
// search.js — готовый скрипт
const input = document.querySelector('#search');
const list = document.querySelector('#results');

fromEvent(input, 'input').pipe(
  map(e => e.target.value),
  debounceTime(300),
  switchMap((term) =>
    fetch(`/api/books?term=${term}`)
    .then((r) => r.json())
  )
).subscribe((books) => {
  list.innerHTML = books
    .map((b) => `<li>${b.title}</li>`).join('');
});
```

Библиотека одна и та же — меняется только место подписки и способ очистки.

Источники

Основные материалы для содержания и дальнейшего изучения.

RxJS documentation

<https://rxjs.dev/>

RxJS course

<https://rxjs-course-avy.web.app/lessons>

Angular Challenges

<https://angular-challenges.vercel.app/>

React-RxJS

<https://react-rxjs.org/>

rxjs-hooks

<https://github.com/LeetCode-OpenSource/rxjs-hooks>

Интерфейс — это значения во времени

RxJS даёт язык, чтобы описать их движение,
конкуренцию и превращение в состояние экрана.

